



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G01

SYSTEMS INTEGRATION

AND ARCHITECTURE 1

SOFTWARE REQUIREMENTS

SPECIFICATION (SRS)

Project Title: StudyMate - Student Study Planner

Prepared By: John Miklos M. Capuras

Date of Submission: July 22, 2026

Version: 3.0

Table of Contents

1. Introduction	3
1.1 Project Title	3
1.2 System Overview	4
1.3 Purpose of the System	4
1.4 Target Users	4
1.5 Problem Statement	4
1.6 Scope of the System	4
1.7 Assumptions and Constraints	5
2. Stakeholders and Users	5
3. Functional Requirements	5
4. Non-Functional Requirements	7
5. Business Rules	8
6. Requirements Traceability Table	9
7. Revision History	10
8. UML Models	
Insert the necessary diagrams for the system:	11
8.1 Entity Relationship Diagram (ERD)	12
8.2 Use Case Models	13
8.3 Activity Diagram — Login, Log Study Session, and Delete Subject Flow	14
9. Initial Project Timeline	17
10. References	17
11. Appendices	18
Appendix A: Data Fields Reference	18
Appendix B: Technology Stack Summary	19

1. Introduction

1.1 Project Title

StudyMate – Student Study Planner

1.2 System Overview

StudyMate is a multi-platform study session logging application consisting of a Spring Boot backend, a ReactJS web application, and an Android Kotlin mobile application. All three components communicate through RESTful APIs and share a centralized relational database (MySQL in production planning; the current deployed backend uses a PostgreSQL-compatible instance on Supabase, accessed through the same Spring Data JPA layer). The backend has been refactored from a layered architecture into a Vertical Slice Architecture (VSA), where each feature — authentication, subject management, session management, and administration — is organized as a self-contained slice spanning its own controller, service, repository, and DTOs.

1.3 Purpose of the System

The purpose of StudyMate is to provide students with a simple, accessible, and dedicated platform to log, view, and manage their study sessions across web and mobile platforms, while giving a designated administrator the ability to oversee registered accounts. The system promotes better time management and more effective study habits.

1.4 Target Users

High school and college students who want to track and manage their study time across multiple subjects, and a single system administrator responsible for account oversight.

1.5 Problem Statement

Students today lack a simple and dedicated platform to record and monitor their study time across multiple courses. General-purpose tools are often too complicated or not designed for academic study tracking, making it difficult for students to assess their own study habits. There is a need for an application that allows students to easily log, view, and manage their study sessions — and their subjects — on both web and mobile platforms, with a reliable backend system to support this and a minimal administrative layer to keep the user base clean.

1.6 Scope of the System

- User registration (with unique username, unique and validated email, and password) and login with secure, token-based authentication
- Creation, viewing, and deletion of academic subjects per user, with duplicate-name prevention
- Logging of study sessions with subject, duration, date, and optional notes

- Viewing of all study sessions recorded by the logged-in user
- Deletion of individual study sessions, and deletion of subjects (blocked when study sessions still reference that subject, to preserve referential integrity)
- Cross-platform access through a ReactJS web application and an Android Kotlin mobile application
- A shared Spring Boot backend, organized by Vertical Slice Architecture, accessed by both platforms via REST API
- A role-based administrator account (seeded at startup, not self-registrable) that can view all registered users and delete a user account, cascading the deletion to that user's subjects and study sessions
- The ability for an existing administrator to create additional administrator accounts, so the ADMIN role can only ever propagate from an already-trusted account

1.7 Assumptions and Constraints

- Users are assumed to have a stable internet connection.
- The system requires the Spring Boot backend to be deployed and accessible before the web and mobile apps can function.
- The mobile application supports Android 8.0 (API Level 26) or higher only.
- The system does not support iOS.
- No offline functionality is provided.
- The production database must be connected and operational for data persistence.
- Only one administrator account exists per deployment, created automatically on backend startup; there is no public admin-registration endpoint.

2. Stakeholders and Users

Stakeholder / User	Role
Student	Registers an account with email, logs in, adds and deletes subjects, logs study sessions, views and deletes sessions.
Administrator	Seeded system account; views the list of all registered users, deletes a user account (cascading to that user's subjects and sessions, but never its own account), and creates additional administrator accounts.
System Administrator	Manages the deployment environment, monitors server uptime, and maintains the database.
Developer	Designs, builds, and maintains the entire system.

3. Functional Requirements

ID	Requirement	Description
FR-001	User Registration	The system shall allow a new user to register by providing a unique username, a unique and validly formatted email address, and a password.
FR-002	User Login	The system shall allow a registered user to log in using their username and password and receive a JWT token upon success.
FR-003	User Logout	The system shall allow an authenticated user to log out and invalidate their session.
FR-004	Add Subject	The system shall allow an authenticated user to add a new academic subject by providing a subject name.
FR-005	View Subjects	The system shall allow an authenticated user to view a list of all their added subjects.
FR-006	Prevent Duplicate Subjects	The system shall prevent a user from adding a subject with the same name twice.
FR-007	Delete Subject	The system shall allow an authenticated user to delete a subject they own, and shall block the deletion — returning a descriptive error — if study sessions still reference that subject.
FR-008	Log Study Session	The system shall allow an authenticated user to log a study session with subject, duration, date, and optional notes.
FR-009	Validate Session Fields	The system shall validate that subject, duration, and date are provided before saving a session.
FR-010	View Study Sessions	The system shall display all study sessions belonging to the currently logged-in user.
FR-011	Delete Study Session	The system shall allow an authenticated user to delete a specific study session by its unique ID.
FR-012	Restrict Data Access	The system shall ensure each user can only access and modify their own subjects and study sessions.
FR-013	Error Handling	The system shall return clear and appropriate error messages for invalid inputs or failed operations.

FR-014	Role-Based Access Control	The system shall embed a role claim (STUDENT or ADMIN) in the JWT and shall restrict all /api/admin/** endpoints to users holding the ADMIN role.
FR-015	Admin View All Users	The system shall allow an authenticated administrator to retrieve and view the list of all registered user accounts.
FR-016	Admin Delete User	The system shall allow an authenticated administrator to delete a user account, cascading the deletion to all subjects and study sessions owned by that user.
FR-017	Admin Create Admin	The system shall allow an authenticated administrator to create a new administrator account; no public or self-service path to the ADMIN role shall exist.

4. Non-Functional Requirements

ID	Category	Requirement
NFR-001	Performance	The system shall respond to standard API requests within 3 seconds under normal network conditions.
NFR-002	Performance	The web and mobile applications shall load their main interface within 5 seconds on a standard broadband connection.
NFR-003	Performance	The backend shall handle concurrent requests from multiple users without significant degradation.
NFR-004	Security	All user passwords shall be stored using BCrypt hashing; plain-text passwords shall never be stored.
NFR-005	Security	All API communication shall occur over HTTPS.
NFR-006	Security	Authentication shall be enforced using JWT tokens to protect all data endpoints.
NFR-007	Security	Users shall only be permitted to access and modify their own data.
NFR-008	Usability	The user interface shall be clean, intuitive, and require no training for a typical student to navigate.
NFR-009	Usability	The system shall provide clear error messages for invalid inputs or failed operations.
NFR-010	Usability	The mobile application shall follow Android Material Design guidelines.
NFR-011	Reliability	The system shall aim for a minimum uptime of 95% during the academic evaluation period.

NFR-012	Reliability	Data entered by users shall be accurately persisted and retrieved without loss or corruption.
NFR-013	Reliability	The system shall handle unexpected errors gracefully rather than crashing.
NFR-014	Compatibility	The mobile application shall support Android OS version 8.0 (API Level 26) or higher.
NFR-015	Compatibility	The web application shall be accessible on modern browsers including Chrome, Firefox, Edge, and Safari.
NFR-016	Maintainability	The backend code shall be organized by Vertical Slice Architecture, grouping each feature's controller, service, repository, and DTOs together.
NFR-017	Maintainability	The web and mobile code shall be organized in modular, readable components to facilitate future updates.
NFR-018	Scalability	The system architecture shall support the addition of new API endpoints and features without requiring a full redesign.
NFR-019	Security	Administrator privileges shall not be obtainable through public registration; the administrator account shall only be created by a startup seeding process.
NFR-020	Data Integrity	The system shall prevent deletion of a subject that still has study sessions logged against it, and shall cascade user deletion to dependent subjects and sessions atomically.
NFR-021	Security	Logout shall be enforced server-side via token revocation (blacklist), not merely by discarding the token on the client.
NFR-022	Data Integrity	Study session creation and deletion operations shall be wrapped in database transactions to guarantee atomic commits.

5. Business Rules

ID	Business Rule
BR-001	A user must register with a unique username. Duplicate usernames are not allowed.
BR-002	A user must be authenticated (logged in) before accessing any feature other than registration.
BR-003	Each subject is associated with a specific user account. Users cannot view or access subjects belonging to other users.

BR-004	A user cannot add two subjects with the same name under the same account.
BR-005	A study session must have a subject, duration, and date before it can be saved. Notes are optional.
BR-006	Each study session belongs to the user who created it. Users cannot view, edit, or delete sessions belonging to other users.
BR-007	Passwords must be hashed using BCrypt before being stored in the database. Plain-text passwords must never be stored.
BR-008	All API requests to protected endpoints must include a valid JWT token. Requests without a valid token must be rejected.
BR-009	The system must run on a reliable internet connection. Offline access is not supported.
BR-010	Only Android devices running OS version 8.0 or higher are supported for the mobile application.
BR-011	A registration email must be required, validly formatted, and unique across all accounts; duplicate emails are rejected the same as duplicate usernames.
BR-012	A subject cannot be deleted while any study session still references it; the user must remove or reassign those sessions first.
BR-013	Only a user holding the ADMIN role may access endpoints under /api/admin/**; all other users receive an access-denied response.
BR-014	When an administrator deletes a user account, all subjects and study sessions owned by that user are deleted as part of the same operation.
BR-015	An administrator shall not be permitted to delete their own account while authenticated as that account.
BR-016	Only an existing administrator may create another administrator account; there is no public or self-service path to the ADMIN role.

6. Requirements Traceability Table

ID	Requirement Name	Use Case	UML Models
FR-001	User Registration	Register account	Use Case, Activity, Sequence
FR-002	User Login	Log in	Use Case, Activity, Sequence
FR-003	User Logout	Log out	Use Case, Sequence
FR-004	Add Subject	Add subject	Use Case, ERD, Class

FR-005	View Subjects	View subjects	Use Case, Sequence
FR-006	Prevent Duplicate Subjects	Add subject	Use Case, Activity
FR-007	Delete Subject	Delete subject	Use Case, ERD, Activity, Sequence
FR-008	Log Study Session	Log study session	Use Case, ERD, Activity, Sequence
FR-009	Validate Session Fields	Log study session	Activity, Sequence
FR-010	View Study Sessions	View study sessions	Use Case, Sequence, ERD
FR-011	Delete Study Session	Delete study session	Use Case, Sequence, ERD
FR-012	Restrict Data Access	All features	Use Case, Sequence
FR-013	Error Handling	All features	Activity, Sequence
FR-014	Role-Based Access Control	Log in, Admin use cases	Use Case, Sequence
FR-015	Admin View All Users	View all users	Use Case, ERD, Sequence
FR-016	Admin Delete User	Delete user account	Use Case, ERD, Activity, Sequence
FR-017	Admin Create Admin	Create admin account	Use Case, Sequence

7. Revision History

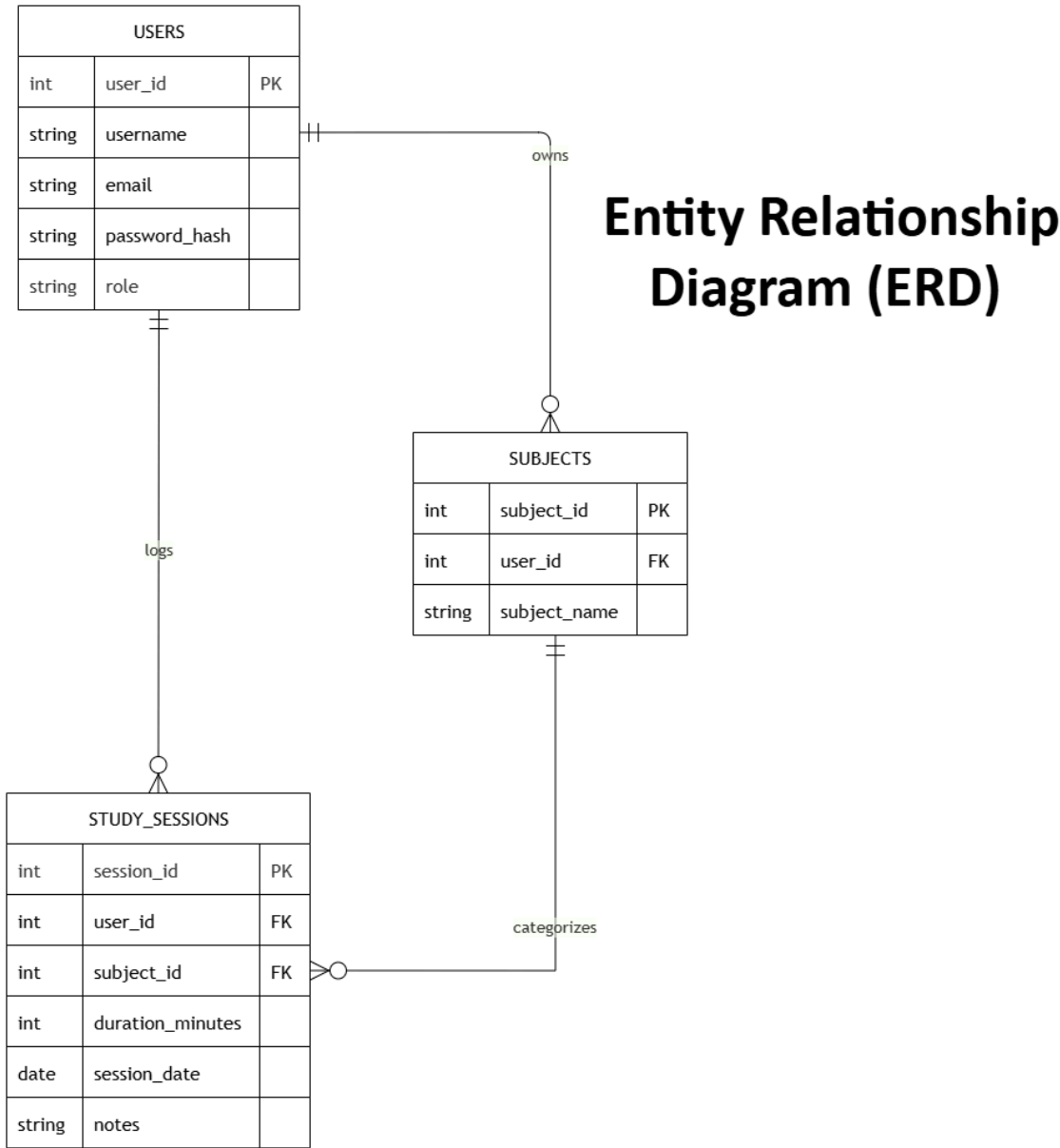
Version	Date	Description
1.0	June 23, 2026	Initial SRS submitted. Included project background, problem statement, objectives, scope, limitations, operating environment, assumptions, and four core functional features without requirement IDs. Non-functional requirements were outlined in paragraph/category format. UML diagrams were marked as "To be Finalized."
2.0	June 30, 2026	Finalized SRS. Restructured document to follow Requirements Engineering format. Added System Overview, Target Users, and Stakeholders and Users section. Rewrote functional requirements with IDs (FR-001 to FR-012). Rewrote non-functional requirements with IDs (NFR-001 to NFR-018). Added Business Rules section (BR-001 to BR-010). Added Requirements Traceability Table. Completed all UML diagrams. Removed Limitations section and integrated constraints into Section 1.7.

3.0	July 22, 2026	Final SRS. Refactored the backend from a layered architecture into a Vertical Slice Architecture (VSA), where each feature — authentication, subject management, session management, and administration — is organized as a self-contained slice spanning its own controller, service, repository, and DTOs. Added Delete Subject (FR-007), with an ownership check and a referential-integrity guard against subjects that still have logged sessions; an email field on registration, validated for format and uniqueness (BR-011); and a role-based Administrator account, seeded at startup, that can view all registered users (FR-015), delete a user account with cascading deletion of that user's subjects and sessions (FR-016), and create additional administrator accounts with no public or self-service path to the ADMIN role (FR-017). Added a safeguard preventing an administrator from deleting their own account while authenticated as it (BR-015). Documented two previously implemented but unrecorded behaviors: server-side logout via token revocation (NFR-021) and transactional atomicity for study session creation/deletion (NFR-022). Added FR-014 to FR-017, NFR-019 to NFR-022, and BR-011 to BR-016. Updated Stakeholders, the Traceability Table, and the Scope of the System accordingly. Rewrote all UML model sections — Use Case Diagram, Entity Relationship Diagram, both Activity Diagrams, and both Sequence Diagrams — in the narrative, reflective style used in v2.0: a short explanation of what each diagram models and why it was designed that way, in front of the precise element list. Diagram figures are not embedded; the author is producing original hand-drawn diagrams from the specifications provided. Retained the Initial Project Timeline, References, and Appendices sections.
-----	---------------	---

8. UML Models

Insert the necessary diagrams for the system:

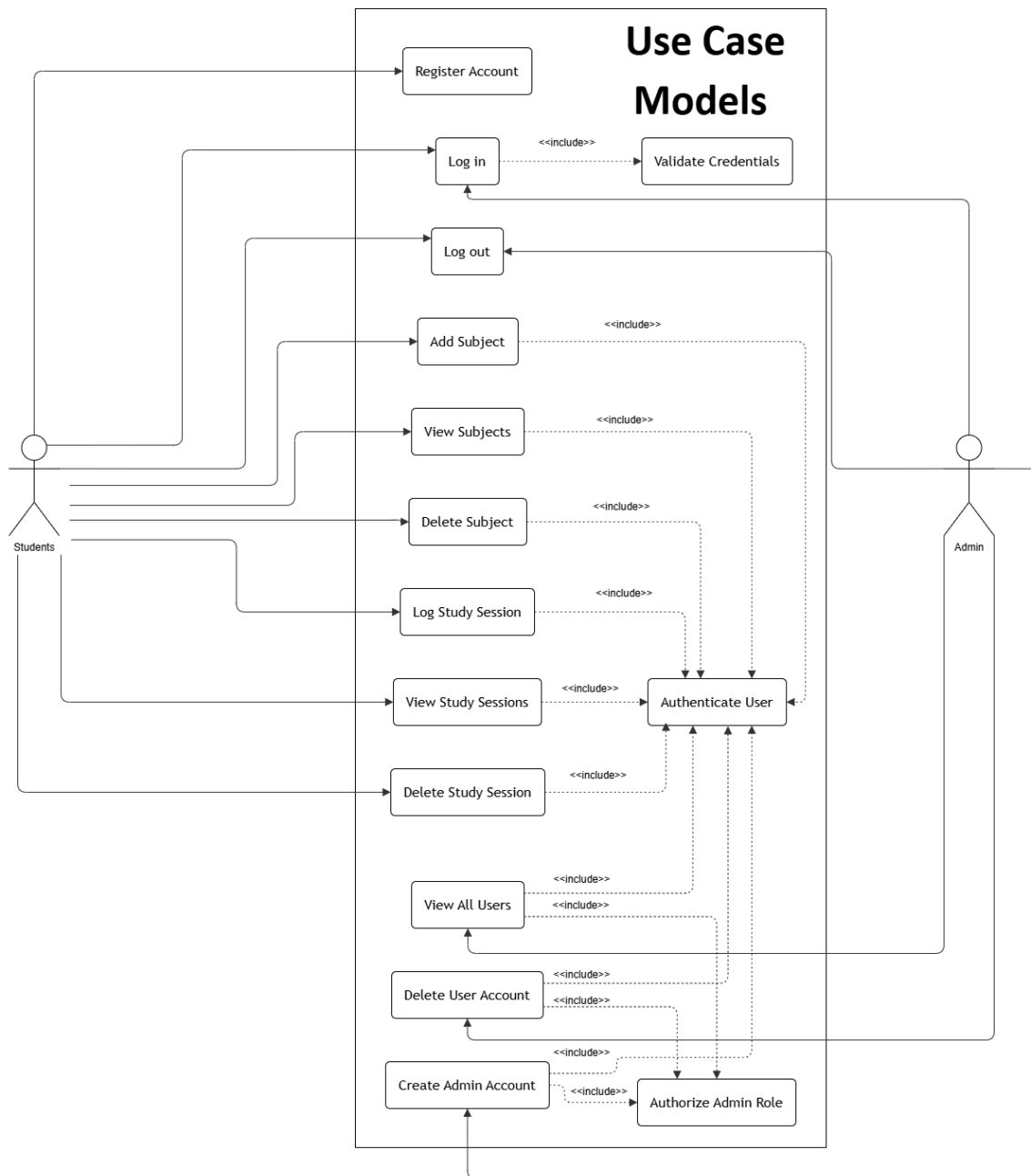
8.1 Entity Relationship Diagram (ERD)



Details:

The ERD consists of three entities: **USERS**, **SUBJECTS**, and **STUDY_SESSIONS**. The **USERS** entity stores user account information, including username, email, password, and role. Each user can own multiple subjects, while each subject can have multiple study sessions. The **SUBJECTS** table stores the subjects created by a user, and the **STUDY_SESSIONS** table records the duration, date, and notes of each study session. These entities are connected through one-to-many relationships using primary and foreign keys to maintain data integrity.

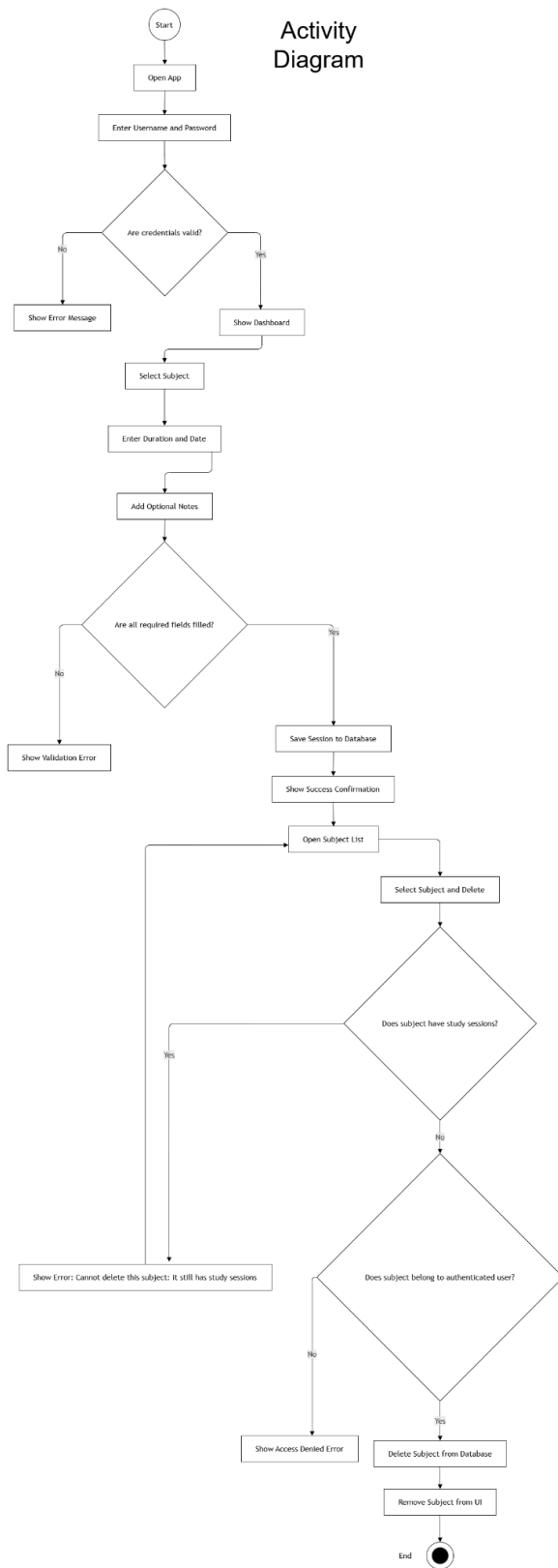
8.2 Use Case Models



Details:

The Use Case Diagram includes two actors: Student and Administrator. Students can register, log in, log out, add subjects, view subjects, delete subjects, log study sessions, view study sessions, and delete study sessions. Administrators can log in, log out, view all users, delete user accounts, and create administrator accounts. Authentication and authorization are handled through supporting use cases to ensure secure access to system functions.

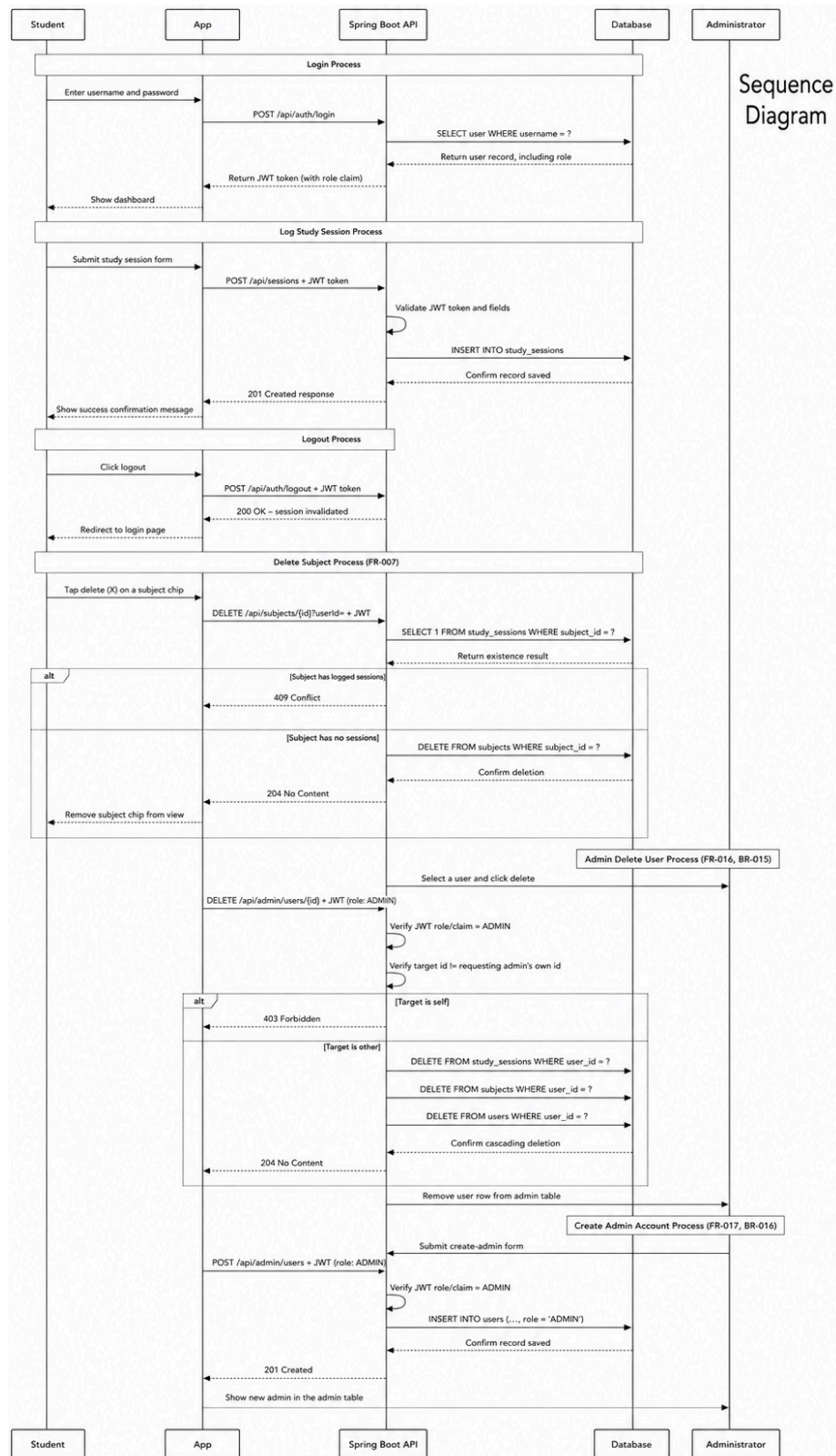
8.3 Activity Diagram — Login, Log Study Session, and Delete Subject Flow



Details:

The Activity Diagram begins when the user opens the application and enters valid login credentials. After successful authentication, the user selects a subject, enters the study session details, and saves the session. The system validates all required fields before storing the information in the database. When deleting a subject, the system first checks whether it contains existing study sessions and verifies ownership before allowing deletion. If validation fails, the user is prompted with an appropriate error message.

8.4 Sequence Diagram — Login, Log Study Session, Logout, Delete Subject, Admin Delete User, and Create Admin



Details:

The Sequence Diagram illustrates the communication among the Student, Administrator, Application, Spring Boot API, and Database. It shows how login requests are authenticated, how study sessions are submitted and saved, how logout requests are processed, how subject deletion requests are validated before removal, and how administrators manage user accounts and create new administrator accounts. The diagram demonstrates the request-and-response flow between each system component.

9. Initial Project Timeline

This is the original six-week execution plan from SRS v1.0, carried forward here for reference. It predates the Vertical Slice Architecture refactor and the administrator-role features added in v3.0–v3.2, so later weeks' actual work expanded beyond what is listed below.

Week	Planned Activity
Week 1	Proposal preparation and submission; project title approval.
Week 2	Requirements refinement; finalization of SRS Version 1.0; initial ERD and use case drafting.
Week 3	UI/UX design for web and mobile applications; database schema design and MySQL setup.
Week 4	Spring Boot backend setup; implementation of user registration and authentication endpoints.
Week 5	Implementation of subject and study session management API endpoints; ReactJS web app development.
Week 6	Android Kotlin mobile app development; system integration testing; deployment to Railway/Render and Vercel; final testing and documentation updates.

10. References

1. Spring Boot Documentation. (2026). Spring Boot Reference Documentation. Retrieved from <https://docs.spring.io/spring-boot/>
2. React. (2026). React – A JavaScript library for building user interfaces. Retrieved from <https://react.dev/>
3. Android Developers. (2026). Android Kotlin Development Documentation. Retrieved from <https://developer.android.com/kotlin>
4. MySQL. (2024). MySQL 8.0 Reference Manual. Oracle Corporation. Retrieved from <https://dev.mysql.com/doc/refman/8.0/en/>

5. IEEE. (1998). IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998). Institute of Electrical and Electronics Engineers.
6. Vercel. (2026). Vercel Platform Documentation. Retrieved from <https://vercel.com/docs>
7. Railway. (2026). Railway Documentation. Retrieved from <https://docs.railway.com/>

11. Appendices

Appendix A: Data Fields Reference

Field	Description	Data Type
User ID	Unique identifier for each registered user.	Integer (Auto-increment)
Username	The unique username chosen by the user during registration.	String (VARCHAR)
Email	The user's unique, validly formatted email address.	String (VARCHAR)
Password	Securely hashed password stored in the database.	String (VARCHAR, BCrypt)
Role	Access level of the account: STUDENT or ADMIN.	String / ENUM
Subject	Name of the academic subject associated with a study session.	String (VARCHAR)
Duration	Length of the study session in minutes.	Integer
Date	Date on which the study session took place.	Date (DATE)
Notes	Optional additional notes about the study session.	String (TEXT, nullable)

The Email and Role fields were not present in v1.0; they were introduced in v3.0 alongside the email-based registration requirement (FR-001, BR-011) and the administrator role (FR-014).

Appendix B: Technology Stack Summary

Layer	Technology	Purpose
Backend	Spring Boot (Java), Vertical Slice Architecture	REST API, business logic, authentication
Web Frontend	ReactJS	Browser-based user interface
Mobile App	Android with Kotlin	Mobile user interface for Android devices
Database	PostgreSQL-compatible instance on Supabase (MySQL 8.0 in original production planning)	Persistent data storage
Backend Hosting	Railway or Render	Cloud deployment of the Spring Boot app
Web Hosting	Vercel	Cloud deployment of the ReactJS app